Smart'Q — Complete Project Documentation

Version: 1.0 (Local development guide)
Product: Smart Queue Management System with ML-powered predictions
Stack: React · Node.js · Express · MongoDB · Python Flask · scikit-learn

---

Table of Contents

1. Problem Statement
2. Solution
3. Project Description
4. System Architecture
5. Project Structure
6. Technology Stack
7. Installation & Startup (Start to End)
8. Environment Configuration
9. Demo Data & Test Accounts
10. Customer Module — Features & Usage
11. Admin Module — Features & Usage
12. Machine Learning — How It Works
13. Analytics & Predictions Screens
14. REST API Overview
15. Real-Time Updates (Socket.IO)
16. Security Features
17. Recent Improvements & Changes
18. Troubleshooting
19. Future Scope
20. Conclusion

---

1. Problem Statement

Public service locations (hospitals, banks, government counters, hiring drives) rely on **physical queues**. This causes:

- Long, visible waiting lines and overcrowding
- No clear view of **position** or **estimated wait time**
- Manual token handling and poor crowd visibility
- Staff difficulty balancing counters and services
- Customers must stay on-site with no remote join option

Traditional token systems generate a number but do not provide **live updates**, **analytics**, or **data-driven scheduling**.

---

2. Solution

Smart'Q digitizes the full queue lifecycle:

Problem	Smart'Q approach
Physical lines	Join from browser; optional QR deep-link to an event

Unknown wait	Rule-based estimates + ML wait-time and peak-hour models
Static boards	Socket.IO live queue updates + polling backup
Admin overload	Dashboards for queues, counters, events, history, analytics
No history	Event archive, queue stats, ML training from join data

Three cooperating services:

1. **React frontend** (port 3000) — customer and admin UI
2. **Node backend** (port 5000) — API, auth, MongoDB, email, sockets
3. **Python ML service** (port 5001) — Random Forest models and training

---

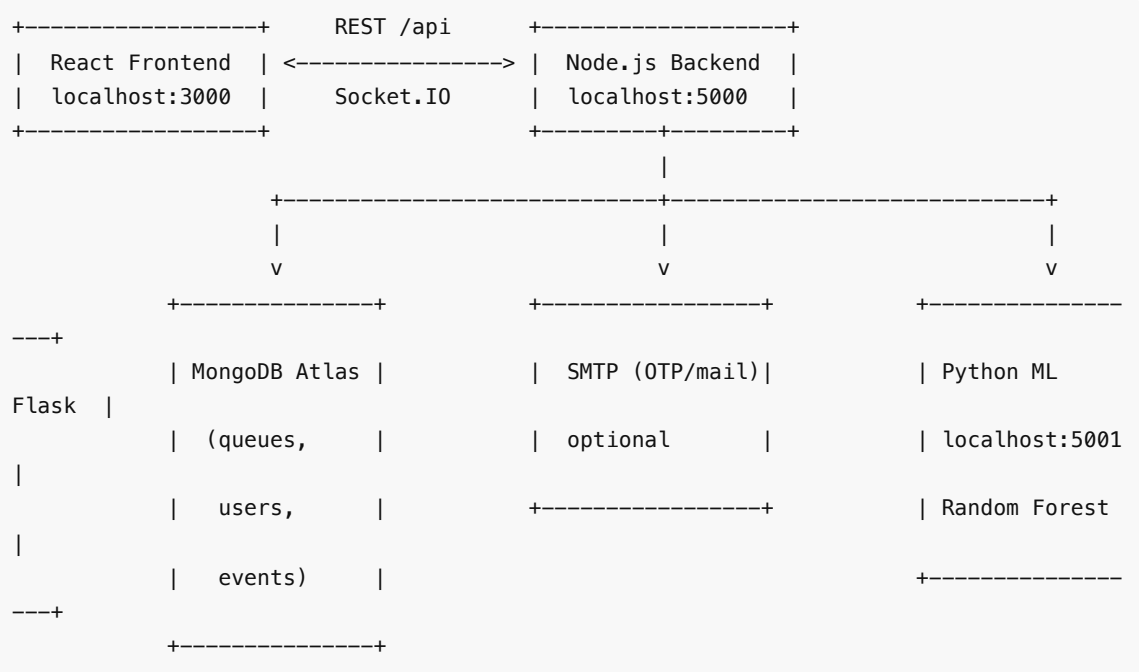
3. Project Description

Smart'Q is a **full-stack smart queue management system** for multi-service environments. Customers authenticate (password or OTP email), browse **events**, join a **service queue**, and track token status in real time. Administrators schedule **events**, manage **tokens** (waiting → serving → completed/cancelled), operate **virtual counters**, view **analytics**, and review **ML predictions**.

The system targets academic and pilot deployments with cloud MongoDB, optional SMTP for OTP/queue emails, and a separable ML microservice.

---

4. System Architecture



Shared Node modules (repo root `src/`): ML config, safe HTTP wrapper to Python, validation, logging — used by backend routes and services.

---

5. Project Structure

```
Smart-Q/
├── README.md
├── docs/
│   └── SmartQ-Complete-Documentation.md   (this document)
├── src/                          # Shared ML integration (Node)
│   ├── config/mlConfig.js
│   ├── services/mlSafeWrapper.js
│   └── utils/mlValidation.js, mlLogger.js
├── frontend/
│   ├── public/index.html         # Meta tags for API base URL (local dev)
│   ├── webpack.config.js
│   ├── .env                      # REACT_APP_API_URL, REACT_APP_ML_URL
│   └── src/
│       ├── App.jsx               # Page routing, session restore on refresh
│       ├── config/api.js         # Local vs production API resolution
│       ├── services/api.js       # Axios client, auth helpers
│       ├── services/queueService.js
│       ├── socket.js
│       └── components/
│           ├── customer/         # Landing, Login, Dashboard, JoinQueue, …
│           ├── admin/            # Dashboard, Queue, Events, Analytics, …
│           └── shared/           # Header, Sidebar, QRCodeDisplay
├── backend/
│   ├── app.js                    # Express app, security, routes, Socket.IO
│   ├── server.js
│   ├── .env                      # Mongo, JWT, SMTP, ML_SERVICE_URL
│   ├── config/db.js
│   ├── controllers/authControllers.js
│   ├── middleware/               # auth, rate limits, login limiter
│   ├── models/                   # User, Queue, Event, OTP, EventHistory, …
│   ├── routes/                   # auth, queue, events, ml, predictions, otp
│   ├── services/
│   │   ├── mlPredictionService.js  # Bundles ML + fallback predictions
│   │   ├── emailService.js
│   │   └── eventCleanupService.js
│   ├── scripts/
│   │   ├── seedDemoUsers.js        # 15 customers + admin
│   │   ├── seedDemoScenarios.js    # Events, queues, history, ML train
│   │   └── createAdmin.js
│   └── ml/
│       ├── ml_service.py           # Flask API
│       ├── models.py               # QueueMLModels (Random Forest)
│       ├── requirements.txt
│       └── saved_models/           # Trained .pkl (gitignored)
└── (legacy docs: ML_SETUP_GUIDE.md, SMTP_CONFIGURATION.md, …)
```

---

6. Technology Stack

Layer	Technologies

Frontend	React 18, Webpack, Axios, Socket.IO client, Lucide icons
Backend	Node.js, Express 5, Mongoose, JWT, bcrypt, Helmet, rate-limit
Database	MongoDB (Atlas or local)
ML	Python 3, Flask, scikit-learn Random Forest, pandas, joblib
Email	Nodemailer (OTP + queue confirmation)
Real-time	Socket.IO (`queue:update` events)

---

7. Installation & Startup (Start to End)

Prerequisites

- Node.js 18+
- Python 3.10+
- MongoDB connection string (`MONGO_URI`)
- (Optional) Gmail app password for SMTP

Step 1 — Backend dependencies

```
cd backend
npm install
cp .env.example .env
# Edit .env: MONGO_URI, JWT_ACCESS_SECRET, ML_SERVICE_URL=http://localhost:5001
```

Step 2 — ML service

```
cd backend/ml
pip install -r requirements.txt
python3 ml_service.py
# Listens on http://localhost:5001
```

Step 3 — Start backend

```
cd backend
npm run dev
# Port 5000
```

Step 4 — Frontend

```
cd frontend
npm install
# .env: REACT_APP_API_URL=http://localhost:5000
npm start
# Opens http://localhost:3000
```

Step 5 — Load demo data (recommended)

```
cd backend
npm run seed:demo
```

This creates users, four sample events, queue rows, event history, and **trains ML models**.

Step 6 — Verify

- Browser: `http://localhost:3000`
- Health: `http://localhost:5000/api/health`
- ML: `http://localhost:5001/health` (should show `trained: true` after seed)

---

8. Environment Configuration

Backend (`backend/.env`)

Variable	Purpose
`PORT`	API port (default 5000)
`MONGO_URI`	MongoDB connection
`JWT_ACCESS_SECRET`	Sign access tokens
`JWT_ACCESS_EXPIRES_IN`	e.g. `8h`
`ML_SERVICE_URL`	`http://localhost:5001` for local
`FRONTEND_ORIGINS`	CORS + Socket.IO origins
`SMTP_*`	OTP and queue emails

Frontend (`frontend/.env`)

Variable	Purpose
`REACT_APP_API_URL`	`http://localhost:5000`
`REACT_APP_ML_URL`	`http://localhost:5001` (reference)

On **localhost**, `frontend/src/config/api.js` forces local API URLs even if HTML meta tags differ.

---

9. Demo Data & Test Accounts

Seed commands

Command	Purpose
`npm run seed:users`	Admin + 15 demo customers only
`npm run seed:demo`	Full scenarios + ML train (resets events/queues)

Admin

Email	Password
`admin@smartq.local`	`AdminPass123!`

Customers (pattern)

Email	Password
`demo.user.01@example.com`	`DemoPass01!`
… through …	…
`demo.user.15@example.com`	`DemoPass15!`

Demo scenarios (after `seed:demo`)

- Hospital OPD, Bank branch, Tech interview, Government ID desk
- Queue tokens in waiting / serving / completed / cancelled
- Two archived rows in Event History

---

10. Customer Module — Features & Usage

10.1 Landing & navigation

- Open `http://localhost:3000`
- Use **Login** or **Join Queue**
- Theme toggle in header (light/dark)

10.2 Registration & login

1. **Sign up:** name, email/mobile, password (min 6 chars + number)
2. **Password login:** email + password
3. **OTP login:** send OTP to email (requires SMTP in backend `.env`)

After login, session is stored (`token`, `authRole`, profile). **Refresh keeps you logged in** (session restore via `/api/auth/me`).

10.3 Join queue

1. Choose an **event** (or scan QR with `?eventId=...`)
2. Enter guest details if prompted
3. Select **service type** (from event)
4. Receive **token number**, wait estimate, crowd level
5. Token saved in `localStorage` for dashboard tracking

10.4 Customer dashboard

- Live **position** and **estimated wait** via Socket.IO + 15s polling backup
- Browse events and join another queue
- Logout clears session

---

11. Admin Module — Features & Usage

Login at **Admin Portal** with `admin@smartq.local`.

Screen	Purpose	How to use
Dashboard	Overview stats, quick links	Open after login
Queue Management	All tokens by status	Move lifecycle: start serving, complete, cancel
Event Scheduler	Create/delete events	Set org type, dates, times, services, token cap
Event History	Archived events	View past stats; delete archive rows
Counters	Per-org/service queues	Call **next token** to start serving
Analytics	Charts + ML summary	Queue trends, actual peak hours, ML 6h outlook
Predictions	Full ML dashboard	Accuracy stats, peak/wait/crowd forecasts
Settings	Admin preferences	Theme, profile placeholders

Queue lifecycle

`waiting` → `serving` → `completed` (or `cancelled`)

Event completion

Event can be **completed** only when all tokens are used and none are waiting/serving.

---

12. Machine Learning — How It Works

12.1 Role in Smart'Q

ML **augments** rule-based logic; if Python is down, the app still queues customers and shows **fallback** analytics.

12.2 Models (Python — `backend/ml/models.py`)

Four **Random Forest** models:

Model	Type	Predicts
Waiting time	Regressor	Minutes to serve given position, time, service
Queue length	Regressor	Expected queue size for hour/day/service
No-show	Classifier	Probability customer won't arrive
Peak hours	Regressor	Queue density for hour/day/service

Features: day of week, hour, month, day of month, encoded service, position in queue (where applicable).

12.3 Training data

- **Manual bulk train:** `POST /train` with array of queue records
- **Admin train:** `POST /api/ml/train` (uses MongoDB queue documents)
- **Auto train on join:** Node calls `POST /queue/joined`; ML buffers joins and retrains every **5** records
- **Demo seed:** `npm run seed:demo` sends ~44 synthetic rows to `/train`

Metadata: R^2 scores stored in `saved_models/metadata.pkl` and exposed on `GET /health` as `model_scores`.

12.4 Inference flow

1. Admin opens Predictions → `GET /api/predictions`
2. Node loads ML health (`trained` , `model_scores`)
3. For each of next **6 hours**, calls `/predict/peak-hours`
4. For next **4 hours**, calls `/predict/waiting-time` with adjusted queue pressure
5. Calls `/suggest/best-time` for optimal visit windows
6. Node **blends** ML output with **real join counts by hour** from MongoDB
7. Applies **relative** High/Medium/Low labels within the forecast window
8. Returns bundle: `peakTimes` , `waitTimePredictions` , `optimalVisitTimes` , `crowdForecast` , `mlModelStats`

12.5 Accuracy metrics (UI)

- **Model Accuracy (wait time):** R^2 of wait-time model × 100 (e.g. 99%)
- **Avg Accuracy:** Mean R^2 across all trained models (e.g. 97%)
- Requires trained models; run `seed:demo` or admin train with ML service up

12.6 ML API (Python, port 5001)

Method	Path	Description
GET	`/health`	Status, trained, model_scores
POST	`/train`	Bulk train
POST	`/predict/waiting-time`	Wait minutes
POST	`/predict/queue-length`	Length forecast
POST	`/predict/no-show`	No-show probability
POST	`/predict/peak-hours`	Density
POST	`/suggest/best-time`	Best hours list
POST	`/queue/joined`	Incremental learning buffer

12.7 Safe wrapper (`src/services/mlSafeWrapper.js`)

- Timeouts, input/output validation, **deterministic fallbacks** if ML fails
- Controlled by `ENABLE_ML` in `mlConfig.js`

---

13. Analytics & Predictions Screens

Analytics

- **Queue Trends (Today):** Histogram of joins by hour from `createdAt`
- **Peak Hours (today, actual):** Top hours by real join count (not ML)
- **Service Popularity:** Top services by active queue rows
- **ML Predictions Summary:** Next 6 hours from `/api/predictions`

Predictions

- **Model Accuracy / Avg Accuracy:** From ML `model_scores`
- **Predictions Today:** Count of forecast slots (6)
- **Peak Time Predictions:** Hour, level, customers, confidence bar
- **Wait Time Predictions:** Now + 3 hours (varies with queue pressure)
- **Crowd Level Forecast:** Cards from peak outlook

---

14. REST API Overview

Base URL: `http://localhost:5000/api`

Area	Examples
Health	`GET /health`
Auth	`POST /auth/login`, `/auth/register`, `/auth/admin/login`, `GET /auth/me`
OTP	`POST /otp/send-otp`, `/otp/verify-otp`
Events	`GET /events`, `POST /events` (admin), delete, complete
Queue	`POST /queue/join`, `GET /queue` (admin), status, lifecycle PUTs
Predictions	`GET /predictions` (admin)
ML proxy	`POST /ml/predict`, `GET /ml/status`, `POST /ml/train`

Most admin routes require header: `Authorization: Bearer <accessToken>`.

---

15. Real-Time Updates (Socket.IO)

- Server attaches Socket.IO to HTTP server in `app.js`
- On queue changes, backend emits **`queue:update`** with waiting/serving lists
- Customer dashboard uses `queueService.js`; admin queue/counter screens use `socket.js`
- Frontend connects to same host as API (`localhost:5000`)

---

16. Security Features

- JWT access tokens with configurable expiry
- bcrypt password hashing
- Helmet, CORS allowlist, rate limiting on `/api`
- Login rate limiter against brute force
- Input sanitization on auth
- Role middleware (`admin` vs `customer`)
- MongoDB sanitization skipped on Express 5 (documented compatibility choice)

---

17. Recent Improvements & Changes

Summary of enhancements applied during project hardening:

Area	Change
Local API	Frontend uses `localhost:5000` on dev; fixed HTML meta override
Auth / refresh	Session restore on page reload; `authRole` + `/auth/me`; removed broken admin refresh that cleared tokens
JWT	Default expiry increased to 8h via `JWT_ACCESS_EXPIRES_IN`
Demo data	`seedDemoUsers.js`, `seedDemoScenarios.js`, npm scripts
Privacy / docs	Removed personal names from README/footer; generic demo emails
Analytics	Filled Peak Hours from real data; fixed chart label layout
ML predictions	Relative peak levels; blended historical joins; varied wait times
ML metrics	`model_scores` in Python metadata; accuracy on Predictions page
Crowd forecast	Populated from peak time bundle
ML health cache	Cleared on each predictions request; longer health timeout

---

18. Troubleshooting

Issue	Action
Login hits Render URL	Restart frontend; check `REACT_APP_API_URL` and hard refresh
Logged out on refresh	Pull latest frontend; ensure token + `/auth/me` restore
ML Predictions N/A accuracy	Start ML on 5001; run `npm run seed:demo`; restart backend
All wait times identical	Restart backend (prediction service update)
Empty events	Run `npm run seed:demo`
OTP not sent	Configure SMTP in `.env`; see `SMTP_CONFIGURATION.md`
Port 5001 in use	`lsof -ti :5001 | xargs kill -9` then restart ML

---

19. Future Scope

- WebSocket-only updates (reduce polling)
- Multi-branch / multi-tenant queues
- SMS/push notifications
- Native mobile apps
- Richer ML features (weather, holidays)

- Remove committed Python `venv` from repo; Docker Compose for all services

---

20. Conclusion

Smart'Q combines **digital queueing**, **real-time operations**, and **machine learning** into one system suitable for hospitals, banks, interviews, and government service counters. With local setup (frontend 3000, backend 5000, ML 5001), demo seed data, and the accounts in Section 9, you can demonstrate the full customer and admin journey end to end.

Document generated for Smart'Q project — queue management with ML insights.